```
$ pytest → 69 passed
```

Testing a Student Result System

Manual and automated testing of a Flask web app with pytest and Selenium WebDriver

Md. Abdullah Mandal

Software Testing Project

THE APPLICATION UNDER TEST

Three features, one written specification

Login

Valid and invalid credentials, clear error messages, protected pages redirect to login.

Add student

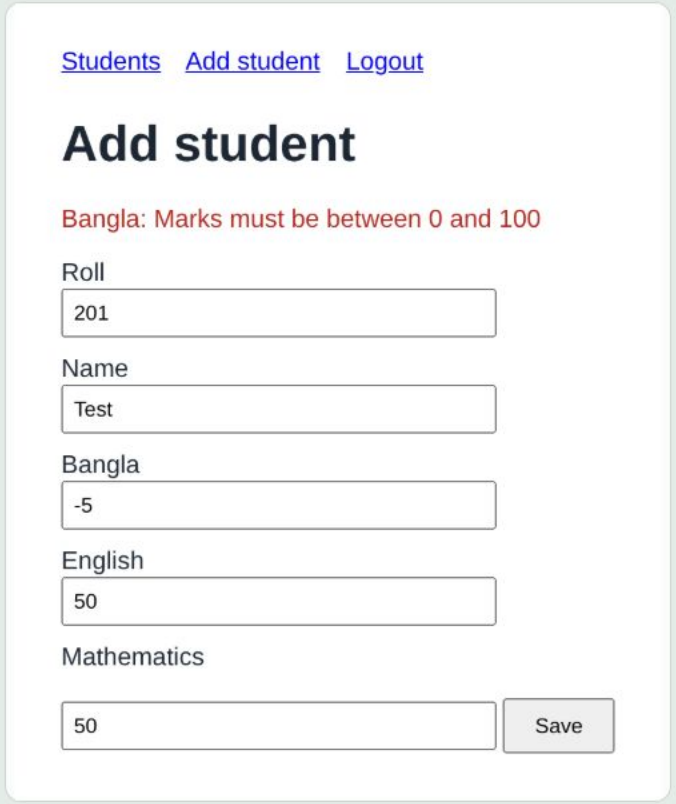
Roll and name required; each mark a whole number from 0 to 100; roll must be unique.

Result

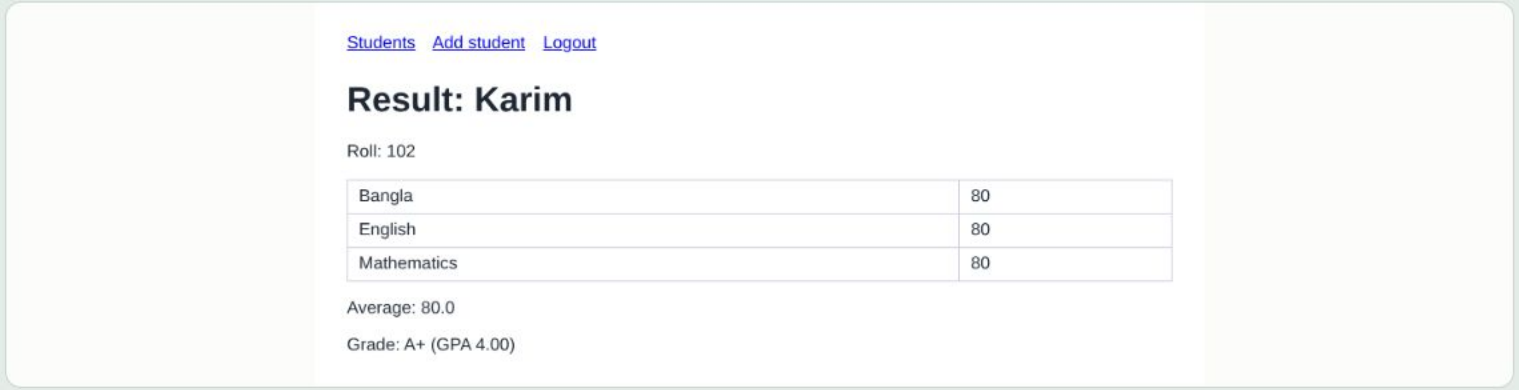
Average of three subjects, then grade and GPA from the Bangladesh UGC grading scale.

THE INTERFACE

What a user sees







Left: invalid mark rejected • Right: student list and a result page (after the fixes)

APPROACH

Following the Software Testing Life Cycle

01
Requirements
Read the rules

02
Test plan
Scope, approach, criteria

03
Test cases
20 manual cases

04
Environment
Python, Chrome, VS Code

05
Execution
69 automated tests

06
Defects
3 defects found

07
Fix and retest
Regression run

08
Closure
Test summary

Documents: test_plan.md • test_cases.csv • bug report template

TEST DESIGN

Choosing inputs where bugs hide

Technique	Idea	Example inputs
Boundary values	Test at, just below and just above every grade edge	80 → A+, 79.99 → A, 40 → D, 39.99 → F
Equivalence partitions	One value from inside each grade band	90, 77, 72 … 20
Negative testing	Invalid input must be rejected	-1, 101, abc, empty, 45.5
Regression testing	Re-run everything after a fix	All 69 tests

20 boundary tests cover all 10 grade bands of the UGC scale.

AUTOMATION

69 automated tests at three levels

48

Unit tests

Grading rules and mark validation, tested directly with pytest.

15

Functional tests

Each matches a manual test case (TC01–TC15), run through the Flask test client.

6

UI tests

End-to-end in a real Chrome browser with Selenium WebDriver.

Plus 20 documented manual test cases in docs/test_cases.csv

SELENIUM

Page Object Model keeps UI tests readable

```
page = AddStudentPage(driver, url).open()
page.fill_and_save("504", "Negative",
                   -10, 50, 50)
assert "Bangla: Marks must be between
  0 and 100" in page.errors()
```

One class per page. Tests read like user actions; element locators live in one file.

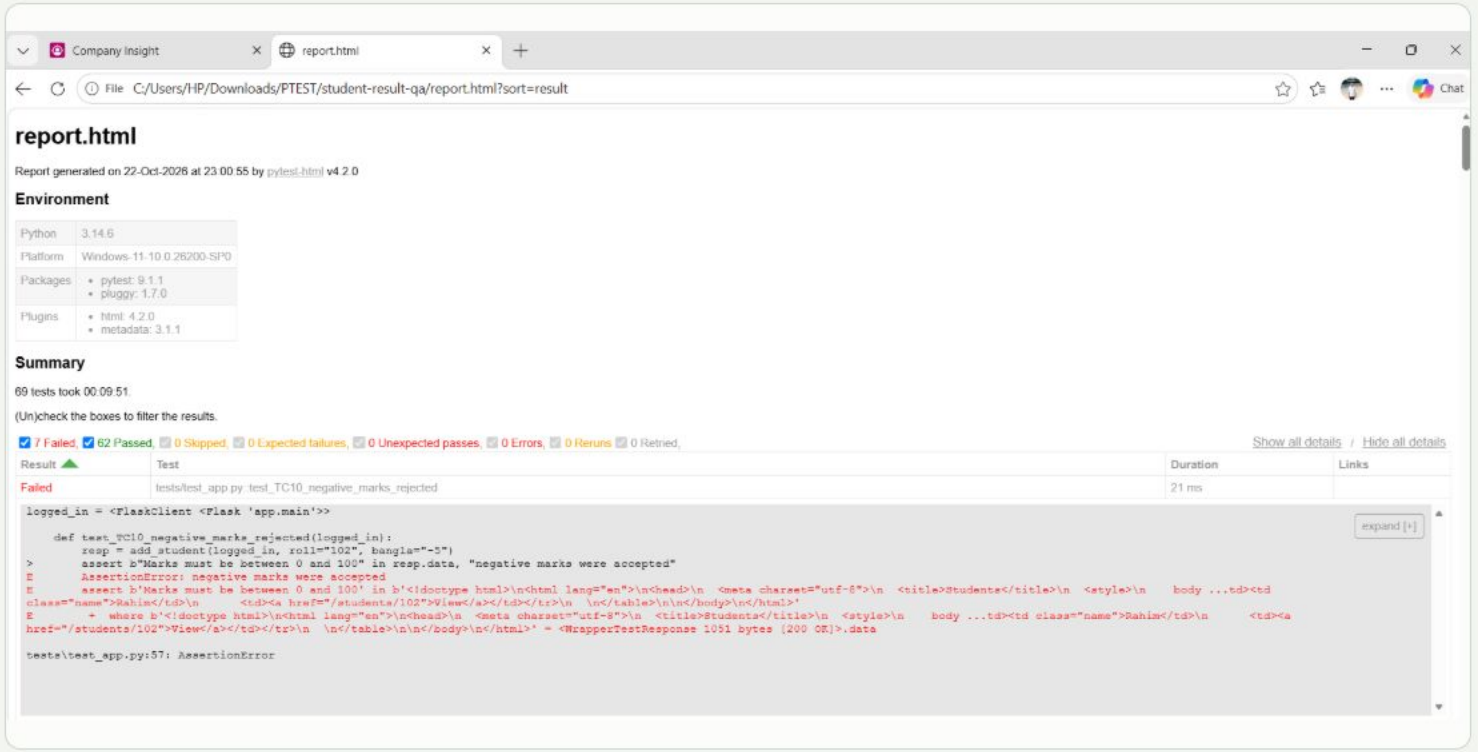
One change, one place. If a button's id changes, only pages.py changes.

Explicit waits. After each submit, the test waits until the next page has loaded.

tests/ui/pages.py • tests/ui/test_ui.py

FIRST RUN

62 passed, 7 failed



The HTML report lists every test with its result and the reason for each failure.

Every failure points at the specification, not at a broken test.

DEFECTS FOUND

Seven failures, three root causes

Bug	What happened	Root cause	Caught by
1	Average of exactly 80 got A, not A+	avg > 80 instead of >=	Boundary test, TC14
2	Negative marks such as -5 were saved	Only the upper limit checked	Negative tests, TC10, UI06
3	Two students saved with the same roll	No duplicate check before insert	TC12

All three are **high severity**: each gives a wrong result or stores invalid data.

FIX AND RETEST

69 / 69

tests passed after the three fixes

`avg ⩾ 80` `mark < 0 or mark > 100` `“Roll already exists”`

Regression run: the fixes work and nothing else broke

LESSONS FROM THE TEST SUITE

Not every failure is a product bug

A timing failure

Three UI tests failed because the browser moved on before login had finished. The server log showed no login request. **Fix:** an explicit wait until the next page loads.

A slow suite

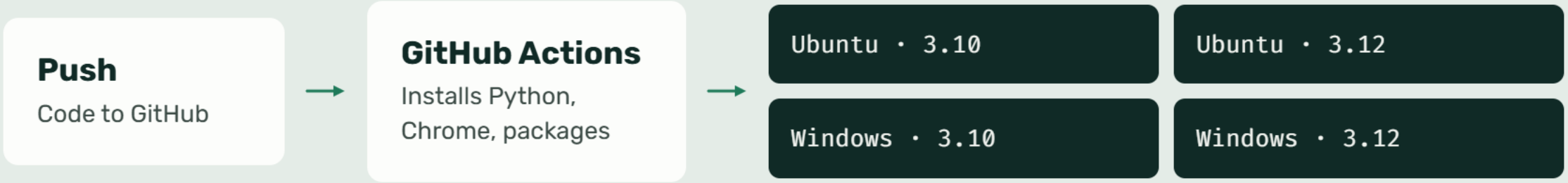
The report showed three UI tests taking about 3 minutes each. Chrome's extra idle connections blocked a single-threaded test server. **Fix:** a threaded server.

10 min 31 s → 30 s full suite, same 69 tests

Measured on the same Windows laptop: 630.89 s before, 29.82 s after

CONTINUOUS INTEGRATION

Every push runs every test, on four setups



Each run saves its HTML test report, so a failure on any setup can be inspected.

.github/workflows/tests.yml

TAKEAWAYS

What this project shows

Tests from the specification find real defects; boundary values found the first one.

A failure needs a root cause before a fix: seven failures, three bugs.

The test suite is software too: waits and setup need debugging.

Tools and techniques

Python • pytest • pytest-html • Selenium WebDriver • Page Object Model • GitHub Actions • BVA • Equivalence partitioning • Negative and regression testing